EECS 3311 Software Design

Github: https://github.com/nurjahan-shiah/Software-Design

Winter 2026

Sections Z

Group 11

Nurjahan Ahmed Shiah

Joseph Ajeigbe

Jessica Buentipo

David Ekeng

Iyinoluwa Olu-Alabi

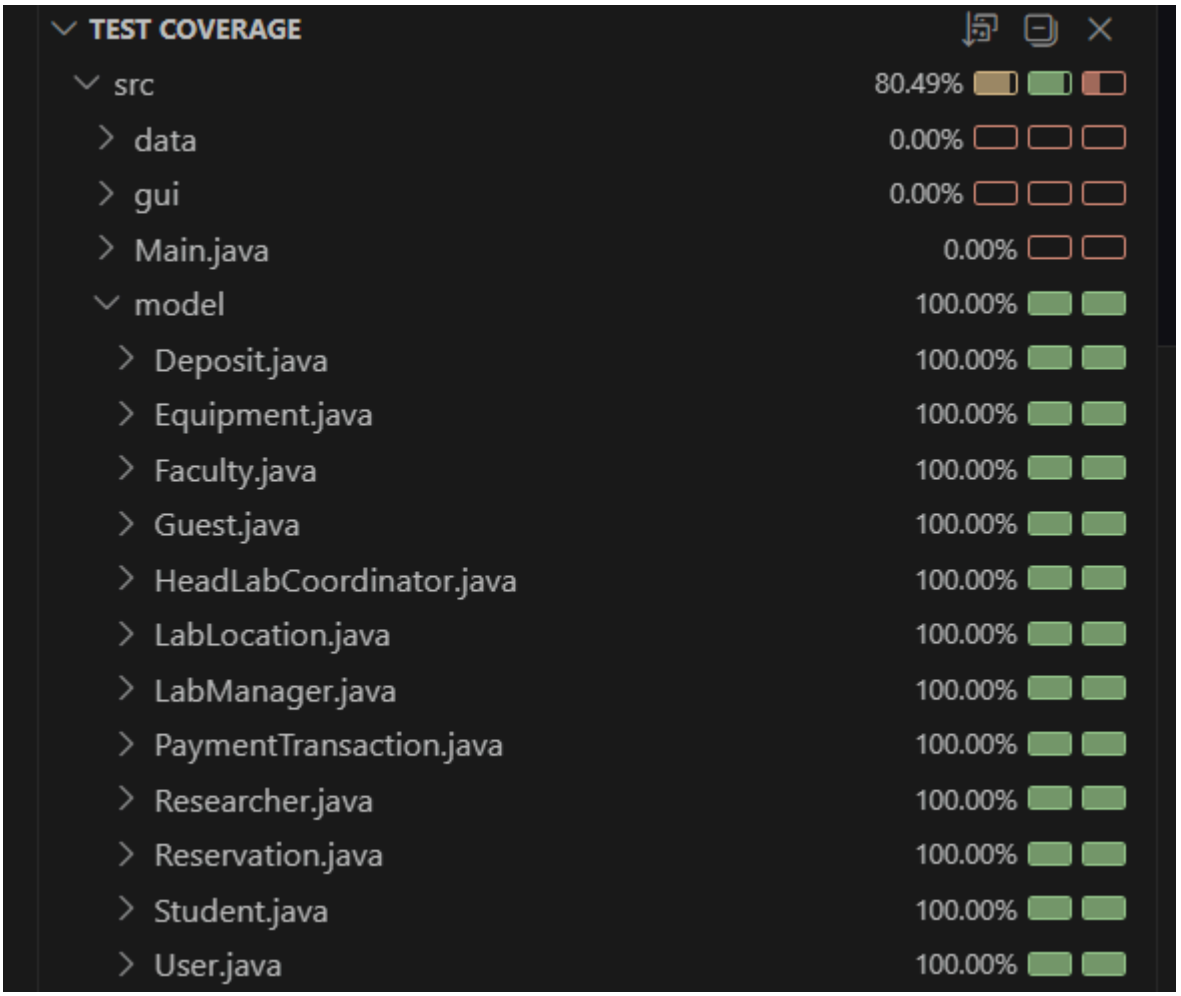
Task 1: Manual JUnit Test Cases

1.1 Overview:

We developed a comprehensive suite of manual JUnit 4 test cases that cover all our non-GUI classes in the project. The test suite was organized into 10 test files, each targeting a specific layer or design pattern of the application.

1.2 Test File Summary

folder: src/model



Folder: src/pattern

Folder: src/test (there are 10 manual tests, 1 folder for AI-Generated tests and one folder for Randoop)

TESTING

TEST EXPLORER

Filter (e.g. text, !exclude, @tag)

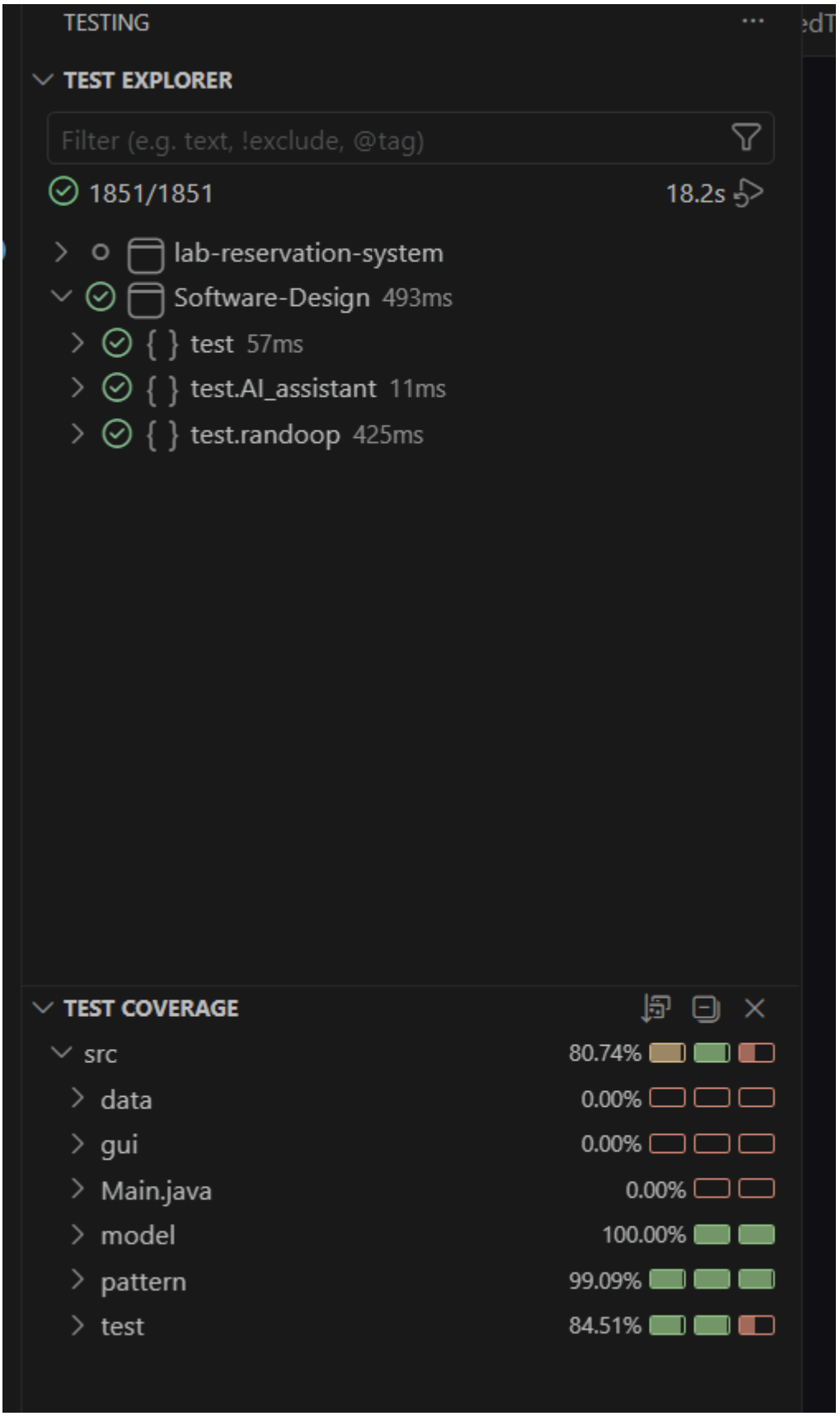
301/301 2.8s

- lab-reservation-system
- Software-Design 633ms
  - { } test 150ms
  - { } test.AI_assistant 69ms
  - { } test.randoop 414ms

TEST COVERAGE

src	80.78%
data	0.00%
gui	0.00%
Main.java	0.00%
model	100.00%
pattern	99.09%
test	84.54%
AI_assistant	100.00%
BookingSystemTests.java	100.00%
CommandTests.java	100.00%
ExtraCoverageTests.java	100.00%
FactoryTests.java	100.00%
ModelTests.java	100.00%
ObserverTests.java	100.00%
randoop	83.39%
SingletonCoordinatorTests.java	100.00%
StateTests.java	100.00%
StrategyTests.java	100.00%
UserTests.java	100.00%

1.3 Code Coverage Results



1.4 Test Design Strategy

Our manual tests were designed based on:

- **State verification**: Each test verifies the system state before and after an operation.
- **Boundary Testing:** Covers default constructors, null values and edge cases.
- **Independence**: Each test case uses fresh instances via @Before setup to avoid state contamination.
- **Completeness**: Every public method in each class has at least one dedicated test.
- **Assertions**: All tests use specific assertEquals/assertTrue/assertFalse checks rather than just checking that no exceptions are thrown.

Task 2: Randoop Auto-Generated Tests

2.1 Overview:

Randoop is a tool that automatically generates JUnit test cases by randomly creating sequences of method calls and recording their outputs. It then generates a regression test that checks that the program behaviour is preserved in future versions.

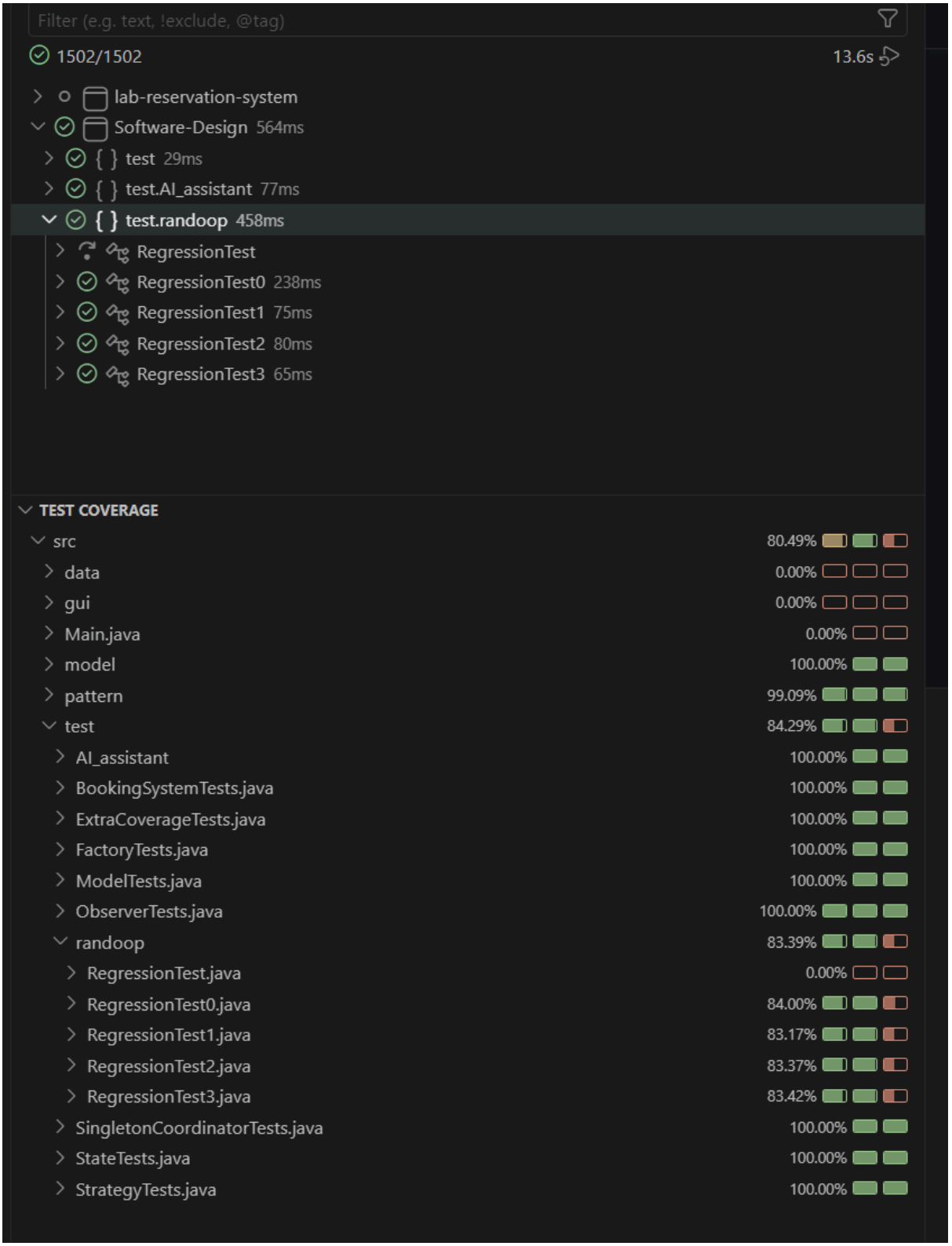
2.2 Running Randoop

Randoop was run on the project using the following command from D2/ directory:

```
java -classpath "out;lib/javacsv.jar;lib/randoop-all-4.2.1.jar" randoop.main.Main gentests --classlist=randoop-classes.txt
--output-limit=500 --time-limit=60
```

The randoop-classes.txt file listed all 27 non-GUI, non-DAO classes to be tested. Randoop generated 4 regression test files (Regression Test0 through RegressionTest3) containing hundreds of auto-generated test classes.

2.3 Randoop Coverage Results



Note: RegressionTest0 showed 0% coverage due to a package declaration issue that prevented it from running in the VS Code test runner. RegressionTest1-3 all achieved approximately 83.39% coverage.

2.4 Observations about Randoop Tests

- Randoop generated a very large number of tests automatically with no manual effort.
- The tests are difficult to read and understand, e.g. method names like test001, test002 provide no semantic information.
- Randoop tests focus on regression; they record current behaviour and fail if it changes, regardless of whether the current behaviour is correct.
- Randoop missed some logical scenarios that require domain knowledge, such as testing the full booking workflow.
- Some Randoop tests created unusual object combinations that would never occur in real usage.

Task 3: AI-Generated Tests

3.1 Overview:

We used Claude (Anthropic, claude-sonnet-4.6) as our AI assistant for test generation. Claude was accessed through the Claude.ai web interface. The AI-generated tests were placed in the src/test/AI_assistant/ folder as AIGeneratedTests.java.

3.2 Prompt Used

Prompt 1: Initial generation request:

"Generate JUnit 4 test cases for the Factory pattern class UserFactory.java in our project. The class creates different User subclasses (Student, Faculty, Researcher, Guest, LabManager, HeadLabCoordinator) based on a string type. Tests should verify: correct type is returned, correct fee rate is returned, null is returned for unknown types, and multiple instances are distinct."

AI Response: Claude generated 11 tests covering all user types, fee rates, instanceof checks, null for unknown type, and distinctness of instances. All tests compiled and passed without modification.

Prompt 2: Command pattern tests:

"Generate JUnit 4 tests for the Command pattern classes: ReserveCommand, CancelCommand, ExtendCommand, ModifyCommand, ForfeitDepositCommand, and ReservationService. Use @Before to create a fresh Reservation and ReservationService before each test. Verify status changes, time updates, and deposit forfeiture."

AI Response: Claude generated 11 command tests covering all 5 commands plus direct service calls. Tests were well-structured with proper @Before setup. All passed without modification.

Prompt 3: Observer and State pattern tests:

"Generate JUnit 4 tests for the Observer pattern (UserNotificationObserver) and State pattern (EquipmentContext with AvailableState, DisabledState, MaintenanceState). For Observer: test that DISABLED and MAINTENANCE notify users, but AVAILABLE does not. Test registering and removing observers. For State: test all state transitions."

AI Response: Claude generated 6 observer tests and 5 state tests. The observer tests were accepted as-is. The state tests needed minor fixes, the initial prompts did not specify the EquipmentContext constructor signature, so Claude assumed wrong parameter types.

Prompt 4: Refinement after errors:

"The EquipmentContext constructor takes (String equipmentID, String initialStatus). Fix the state tests to use: new EquipmentContext(\"EQU001\", \"AVAILABLE\")."

AI Response: Claude immediately corrected all 5 state tests. This illustrates how prompt refinement is needed when the AI lacks exact API signatures.

3.3 Accepted, Modified, and Rejected Tests

3.4 Examples of Failed and Successful Tests

Failed Test Example (required modification):

```
// AI generated (WRONG - assumed no-arg constructor)
        new EquipmentContext().disable();
// Fixed version
        new EquipmentContext("EQU001", "AVAILABLE").disable();
```

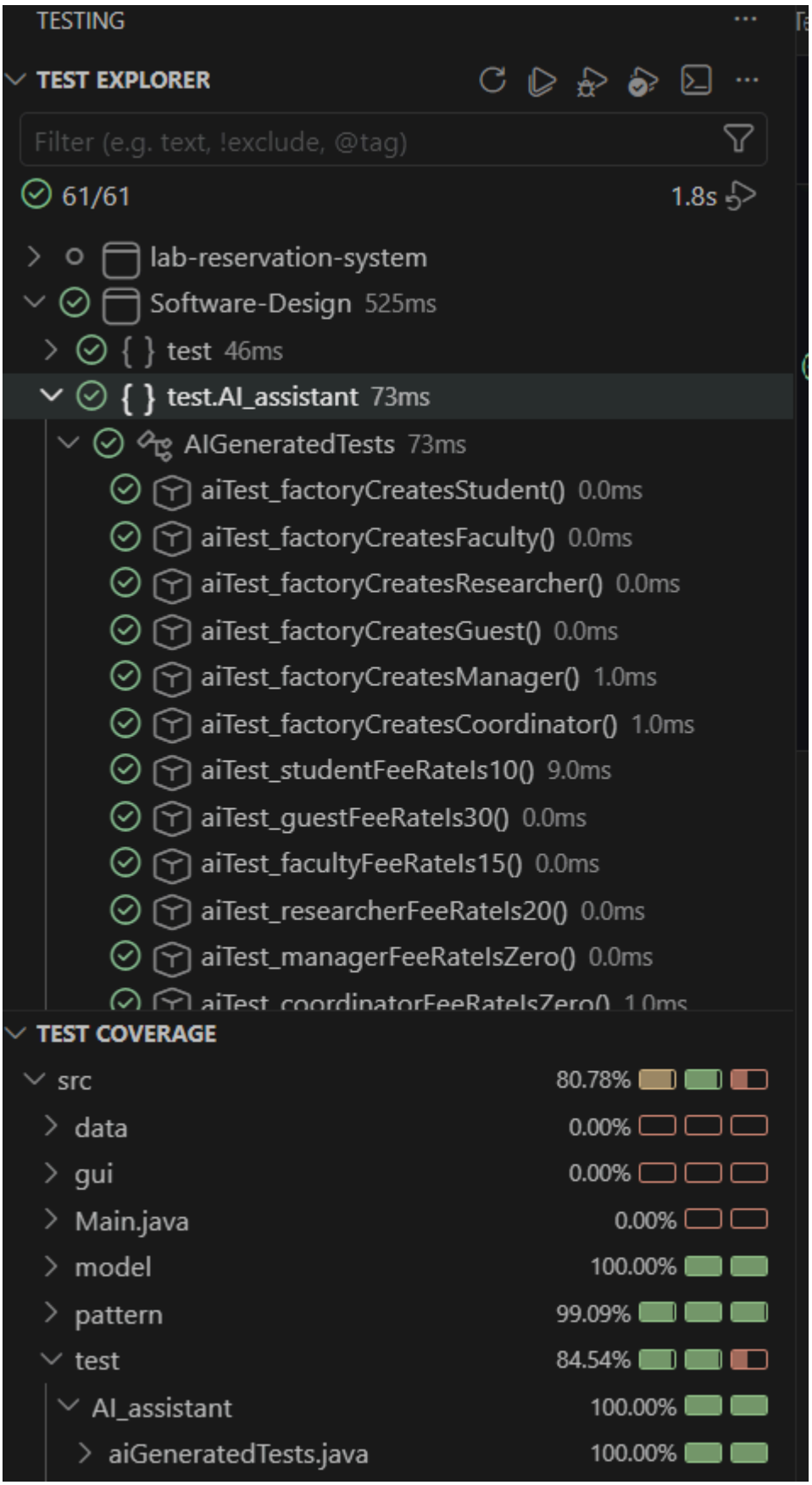
Successful Test Example:

```
// AI generated (CORRECT - accepted as-is)
@Test
public void aiTest_factoryCreatesStudent() {
        UserFactory factory = new UserFactory();
```

```
User user = factory.getUserType("STUDENT");
        assertNotNull(user);
        assertTrue(user instanceof Student);
}
```

3.5 AI Coverage Results

The AI-generated tests in AIGeneratedTests.java achieved approximately 100% coverage of the classes they targeted, covering all pattern classes and model classes with meaningful assertions.



3.6 Reflection on AI-Assisted Testing

Strengths of AI-assisted test generation:

- Very fast; generated 61 tests in minutes vs hours manually
- Good at covering all methods and constructor variants
- Produced clean, readable test names and structure
- Correctly identified edge cases like null strategy in PaymentProcessor

Limitations:

- Required exact API signatures, made assumptions when not provided
- Did not generate meaningful integration/workflow tests (e.g. full booking lifecycle)
- Some tests were redundant duplicates of each other
- Could not understand domain-specific business rules without detailed prompting

Lessons learned:
AI tools are most effective when given precise class signatures and clear behavioural specifications. Prompt refinement is essential; the first prompt rarely produces perfect results.
Manual review is necessary to catch incorrect assertions and ensure meaningful test coverage.

Task 4: Comparison of Testing Approaches

4.1 Code Coverage Comparison

Approach	**Coverage Achieved**	**Classes Covered**	**Notes**
Manual JUnit	Model: 100% Pattern: 99.09%	All 30 non-GUI classes	achieved almost the utmost coverage because they were written with full knowledge of the class internals.
Randoop	83.39%	Pattern + Model classes	solid coverage automatically, but missed some edge cases.
AI-generated	100%	All pattern + model classes	Highest coverage. Targeted every method

4.2 Mutation Score Comparison

4.2.1 Automated Mutation Score (PITest)

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
30	99%	345/346	85%	133/157	85%	133/156

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
model	11	100%	140/140	100%	45/45	100%	45/45
pattern.command	6	100%	50/50	100%	16/16	100%	16/16
pattern.factory	1	100%	14/14	100%	12/12	100%	12/12
pattern.observer	1	100%	15/15	75%	3/4	75%	3/4
pattern.singleton	2	100%	40/40	75%	18/24	75%	18/24
pattern.state	4	98%	61/62	72%	28/39	74%	28/38
pattern.strategy	5	100%	25/25	65%	11/17	65%	11/17

4.2.2 Manual Mutation Score

Mutation	class	method	change	result	Killed by
1	deposit.java	constructor	Before: this.isForfeited = true; After: *this.isForfeited = false;*	KILLED	testDepositForfeit
2	deposit.java	forfeit()	Removed body	KILLED	testDepositForfeit
3.	deposit.java	isForfeited()	return isForfeited → *return !isForfeited*	KILLED	testDepositInitialState
4.	deposit.java	getAmount()	return amount → *return 0*	KILLED	testDepositSetAmount
5.	deposit.java	setAmount()	Removed body	KILLED	testDepositSetAmount
6.	LabLocation.java	getBuildingName()	return buildingName → *return “”*	KILLED	testLabLocationConstructorAndGetters
7.	LabLocation.java	getRoomNumber()	return roomNumber → *return “”*	KILLED	testLabLocationConstructorAndGetters
8.	LabLocation.java	getLocationDetails()	return buildingName + “ - Room ” + roomNumber →	KILLED	testLabLocationGetLocationDetails

			return “”		
9.	LabLocation.java	toString()	return buildingName + “ , ” + roomNumber → *return “”*	KILLED	testLabLocationToString
10.	LabLocation.java	setBuildingName()	Removed body	KILLED	testLabLocationSetters

4.3 Readability Comparison

Criteria	**Manual JUnit**	**Randoop**	**AI-Generated**
Test naming	Descriptive (e.g. testDepositForfeit, cancelTest)	Generic (test001, test002)	Descriptive (aiTest_factoryCreatesStudent)
Structure	well-organized by class and concern with comments	Flat, no organization	organized by pattern with comments
Assertions	Meaningful, specific values checked	Often trivial, checks not-null only	Good, specific values checked
Maintability	Easy to update when code changes	Very hard; no indication of what is tested	Moderate; clear names, but some redundancy
Overall	**High**	**Low**	**High**

Manual and AI-Generated tests are significantly more readable than Randoop tests. Randoop's auto-generated test names and absence of comments make it very difficult to understand what each test is verifying. We have followed a clear naming convention in Manual Testing, describing the class, method, and expected behaviour, making them easy to maintain and extend.

4.4 Usefulness Comparison

Criteria	Manual JUnit	Randoop	AI-Generated
Catches real faults	Yes, targeted at business logic	Partially, catches regressions	Yes, good at business logic
Covers edge cases	Yes, explicitly designed	Sometimes, random generation	Partially requires good prompts
Tests workflows	Yes, full booking lifecycle tests	No, individual method calls only	Partially, some workflow tests
Non-trivial assertions	Yes	Sometimes	Yes
Domain Knowledge	Yes, as tests are created with knowledge of the domain	No, does not create tests with knowledge of intended behaviour	Partially, prompts require more detail to generate accurate tests

4.5 Summary and Conclusion

Approach	Coverage	Mutation Score	Overall
Manual JUnit	~99%	85%	Best effort/quality ratio
AI-Generated	~100%	47%	Best for time efficiency
Randoop	~83%	46%	Best for regression only

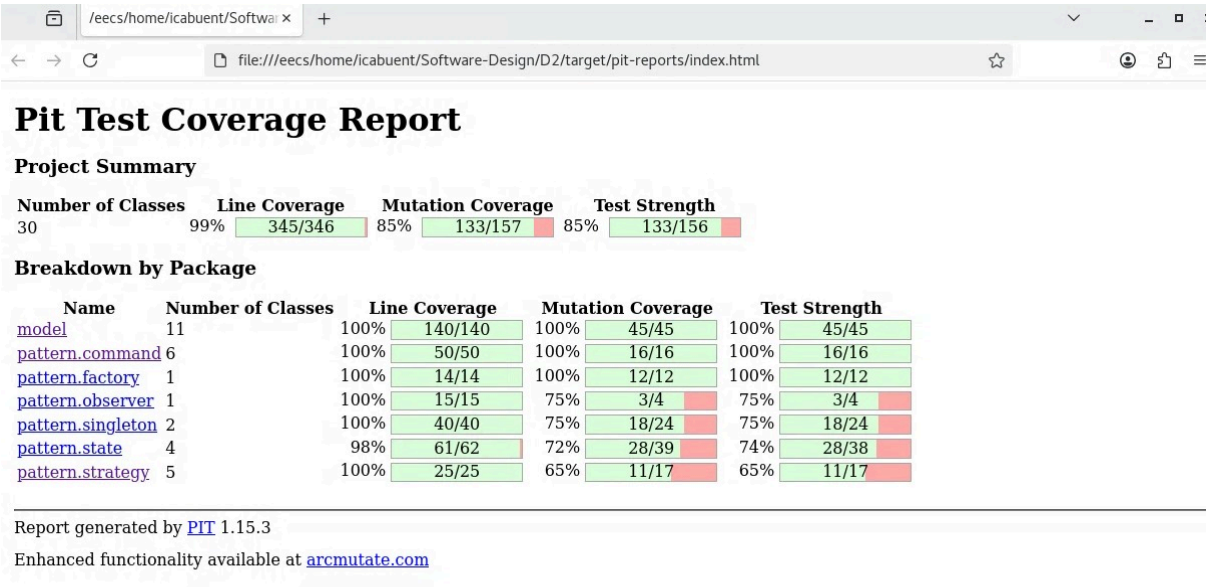
Manual JUnit Tests were the most useful for ensuring correctness and validating business logic. Because these tests were written with full knowledge of the system design, they were able to target specific behaviours, edge cases, and workflows (e.g., reservation lifecycle, deposit handling). They were also the most effective at killing meaningful mutations, as seen in the mutation testing results. However, they required significant development time and effort.

Randoop Tests were most useful for quick regression testing with minimal effort. They generated a large number of tests automatically and achieved reasonable coverage. However, their usefulness is limited because they do not understand intended behaviour, they only record existing behaviour. As a result, they may validate incorrect logic and miss specific scenarios. Additionally, their poor readability and lack of structure make them difficult to maintain or extend.

AI-Generated Tests provided the best balance between efficiency and quality. They were generated quickly while still producing readable, well-structured tests with meaningful assertions. AI was particularly effective at covering all methods and identifying common edge cases. However, their usefulness depends heavily on prompt quality and accurate API information. Without precise inputs, the AI made incorrect assumptions (e.g., constructor signatures) and required manual correction. They also lacked deeper system-level or integration testing without explicit prompting.

References for PIT Test Coverage Report

Manual



/eecs/home/icabuent/Softwa ×

file:///eecs/home/icabuent/Software-Design/D2/target/pit-reports/index.html

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
30	99%	345/346	85%	133/157	85%	133/156

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
model	11	100%	140/140	100%	45/45	100%	45/45
pattern.command	6	100%	50/50	100%	16/16	100%	16/16
pattern.factory	1	100%	14/14	100%	12/12	100%	12/12
pattern.observer	1	100%	15/15	75%	3/4	75%	3/4
pattern.singleton	2	100%	40/40	75%	18/24	75%	18/24
pattern.state	4	98%	61/62	72%	28/39	74%	28/38
pattern.strategy	5	100%	25/25	65%	11/17	65%	11/17

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com

Randoop

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
30	81%	281/347	46%	72/155	63%	72/115

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
model	11	96%	135/140	93%	42/45	93%	42/45
pattern.command	6	80%	40/50	25%	4/16	27%	4/15
pattern.factory	1	7%	1/14	0%	0/12	100%	0/0
pattern.observer	1	50%	7/14	0%	0/4	0%	0/3
pattern.singleton	2	61%	25/41	40%	10/25	83%	10/12
pattern.state	4	79%	50/63	28%	10/36	40%	10/25
pattern.strategy	5	92%	23/25	35%	6/17	40%	6/15

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com

AI-generated

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
30	66%	228/346	47%	74/157	69%	74/108

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
model	11	39%	54/140	27%	12/45	67%	12/18
pattern.command	6	100%	50/50	56%	9/16	56%	9/16
pattern.factory	1	93%	13/14	100%	12/12	100%	12/12
pattern.observer	1	87%	13/15	75%	3/4	75%	3/4
pattern.singleton	2	68%	27/40	46%	11/24	79%	11/14
pattern.state	4	74%	46/62	49%	19/39	70%	19/27
pattern.strategy	5	100%	25/25	47%	8/17	47%	8/17

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com